implement C function which will take x, y coordinates of a starting point on Mercator projection map, heading in degrees, distance in meters and a scale factor and then return x, y coordinates of a destination point

Yes — the clean way is: convert the Mercator x, y start point back to latitude/longitude, move by the given heading and distance on the sphere, then project the destination latitude/longitude back to Mercator x, y. The Mercator forward/inverse formulas and destination-point formulas are standard and fit this use case well.[1] [2]

C function

The function below assumes a spherical Mercator projection with Earth radius 6378137 m, which is the usual Web Mercator radius, and treats `scale` as a multiplier applied to projected meters when returning map coordinates. Mercator uses $x = R\lambda$ and $y = R\ln(\tan(\pi/4 + \phi/2))$, while the destination point from start latitude/longitude, distance, and bearing uses the standard great-circle formulas with $\delta = d/R$.[2] [1]

```
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

typedef struct {
    double x;
    double y;
} PointXY;

static double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

static double normalize_lon(double lon_rad) {
    while (lon_rad > M_PI)  lon_rad -= 2.0 * M_PI;
    while (lon_rad < -M_PI) lon_rad += 2.0 * M_PI;
    return lon_rad;
}

/*
 * start_x, start_y  - Mercator projected coordinates of start point
 * heading_deg       - bearing clockwise from north
```

```
 * distance_m        - travel distance in meters
 * scale             - map scale factor applied to projected coordinates
 *
 * Assumptions:
 * - Spherical Mercator / Web Mercator
 * - Input x,y are in "scaled Mercator meters":
 *       x = scale * R * lon
 *       y = scale * R * ln(tan(pi/4 + lat/2))
 *
 * Returns destination x,y in the same scaled coordinate system.
 */
PointXY mercator_destination(double start_x,
                             double start_y,
                             double heading_deg,
                             double distance_m,
                             double scale)
{
    const double R = 6378137.0;
    PointXY out = {0.0, 0.0};

    if (scale == 0.0) {
        return out;
    }

    /* Unscale projected coordinates back to Mercator meters */
    double mx = start_x / scale;
    double my = start_y / scale;

    /* Inverse Mercator: x,y -> lon,lat */
    double lon1 = mx / R;
    double lat1 = 2.0 * atan(exp(my / R)) - M_PI / 2.0;

    /* Great-circle destination formula */
    double brng = deg2rad(heading_deg);
    double delta = distance_m / R;

    double sin_lat1 = sin(lat1);
    double cos_lat1 = cos(lat1);
    double sin_delta = sin(delta);
    double cos_delta = cos(delta);
    double sin_brng = sin(brng);
    double cos_brng = cos(brng);

    double lat2 = asin(sin_lat1 * cos_delta +
                       cos_lat1 * sin_delta * cos_brng);

    double lon2 = lon1 + atan2(sin_brng * sin_delta * cos_lat1,
                               cos_delta - sin_lat1 * sin(lat2));

    lon2 = normalize_lon(lon2);

    /* Forward Mercator: lon,lat -> x,y */
    double dest_mx = R * lon2;
    double dest_my = R * log(tan(M_PI / 4.0 + lat2 / 2.0));

    /* Re-apply scale */
```

```
    out.x = dest_mx * scale;
    out.y = dest_my * scale;

    return out;
}
```

How `scale` is used

If your map coordinates are stored as projected meters directly, pass `scale = 1.0`. If your map uses pixels or some other linear unit derived from Mercator meters, `scale` should be the multiplier between Mercator meters and your coordinate system, because Mercator's projected coordinates are linear in x and y.[1]

For example, if your map uses `screen_x = mercator_x * 0.25` and `screen_y = mercator_y * 0.25`, then call the function with `scale = 0.25`, and the returned point will be in the same screen-space coordinate system. This works because the function removes the scale before geographic math and reapplies it after projecting the destination.[1]

Important note

If you actually want a constant-heading path on the Mercator map, that is a rhumb line, not a great-circle path. On Mercator, rhumb lines are straight lines and use a different destination formula than the shortest-path great-circle formulas used above.[2] [1]

If you want, I can also give you:

- a rhumb-line version for Mercator navigation,
- a WGS84 ellipsoidal version with better accuracy,
- or a version that takes/returns latitude and longitude too.

⁂

give me a WGS84 ellipsoidal version with better accuracy,

Here is a WGS-84 ellipsoidal version of your function that first converts Mercator map coordinates to geographic latitude/longitude, then computes the destination point along a geodesic on the WGS-84 ellipsoid, and finally maps that back to Mercator x, y with your scale factor.[11] [12] [13]

Key features:

- Uses WGS-84 ellipsoid parameters $a \approx 6378137\ \mathrm{m}$, $f \approx 1/298.257223563$.[12]
- Applies the "direct" geodesic solution (Vincenty-style) for the destination from latitude/longitude, distance, and bearing.[13] [14]
- Uses the Mercator forward/inverse formulas as on the ellipsoid, rescaled by your map `scale`.[15] [11]

C function (WGS-84 ellipsoidal)

```c
#include <math.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

typedef struct {
    double x;
    double y;
} PointXY;

static double deg2rad(double deg) {
    return deg * M_PI / 180.0;
}

static double rad2deg(double rad) {
    return rad * 180.0 / M_PI;
}

static double normalize_lon(double lon_rad) {
    while (lon_rad > M_PI)  lon_rad -= 2.0 * M_PI;
    while (lon_rad < -M_PI) lon_rad += 2.0 * M_PI;
    return lon_rad;
}

/*
 * Approximate inverse Mercator on WGS84 ellipsoid.
 *
 * xi = x / scale = a * lon
 * yi = y / scale = a * ln( tan( pi/4 + lat/2 ) )
 * where a = 6378137 m is the WGS84 semi-major axis.
 *
 * This returns latitude/longitude in radians.
 */
void mercator_to_geo(double xi, double yi,
                     double a, double *lat_out, double *lon_out)
{
    double lon = xi / a;
    double phi = 2.0 * atan(exp(yi / a)) - M_PI / 2.0;
    *lon_out = lon;
    *lat_out = phi;
}

/*
 * Approximate forward Mercator on WGS84 ellipsoid.
 *
 * Returns Mercator coordinates in meters (before scaling).
 */
void geo_to_mercator(double lat, double lon,
                     double a,
                     double *mx_out, double *my_out)
{
    *mx_out = a * lon;
    *my_out = a * log(tan(M_PI / 4.0 + lat / 2.0));
```

```
}

/*
 * WGS-84 ellipsoidal geodesic direct solution.
 *
 * From start (lat1, lon1) in radians, distance s in meters,
 * and bearing brng_rad, compute destination (lat2, lon2) in radians.
 *
 * Uses approximate WGS-84 ellipsoid parameters.
 */
void ellipsoid_direct(double lat1, double lon1,
                      double s, double brng_rad,
                      double *lat2_out, double *lon2_out)
{
    const double a = 6378137.0;          /* WGS-84 semi-major axis */
    const double f = 1.0 / 298.257223563;/* WGS-84 flattening */
    const double b = a * (1.0 - f);      /* semi-minor axis */
    const double ep2 = (a*a - b*b) / (b*b);  /* e'² = (a²-b²)/b² */

    double sin_lat1 = sin(lat1);
    double cos_lat1 = cos(lat1);

    double sin_brng = sin(brng_rad);
    double cos_brng = cos(brng_rad);

    double sin2_brng = sin_brng * sin_brng;
    double cos2_brng = cos_brng * cos_brng;

    double tan_lat1 = sin_lat1 / cos_lat1;
    double U1 = atan((1.0 - f) * tan_lat1);

    double sin_U1 = sin(U1);
    double cos_U1 = cos(U1);

    double sigma1 = atan2(sin_U1, cos_U1 * cos_brng);
    double sin_alpha = cos_U1 * sin_brng;
    double cos2_alpha = 1.0 - sin_alpha*sin_alpha;

    double u2 = cos2_alpha * ep2;

    double A = 1.0 + u2/16384.0*(4096.0 + u2*(-768.0 + u2*(320.0 - 175.0*u2)));
    double B = u2/1024.0*(256.0 + u2*(-128.0 + u2*(74.0 - 47.0*u2)));

    double sigma = s / (b * A);

    double sigma_prev;
    double cos2_sigma_m, sin_sigma, cos_sigma;
    int iter = 0;

    do {
        sigma_prev = sigma;

        cos2_sigma_m = cos(2.0*sigma1 + sigma);
        sin_sigma = sin(sigma);
        cos_sigma = cos(sigma);
```

```
        double delta_sigma = B * sin_sigma * (
            cos2_sigma_m + B/4.0 * (
                cos_sigma*(-1.0 + 2.0*cos2_sigma_m*cos2_sigma_m)
                - B/6.0 * cos2_sigma_m * (-3.0 + 4.0*sin_sigma*sin_sigma)
                  *(-3.0 + 4.0*cos2_sigma_m*cos2_sigma_m)
            )
        );

        sigma = s / (b * A) + delta_sigma;
    } while (fabs(sigma - sigma_prev) > 1e-12 && ++iter < 100);

    sin_sigma = sin(sigma);
    cos_sigma = cos(sigma);

    double sinU1_cos_sigma = sin_U1 * cos_sigma;
    double cosU1_sin_sigma = cos_U1 * sin_sigma;

    double sin_lat2 = sinU1_cos_sigma + cosU1_sin_sigma * cos_brng;

    double lambda = atan2(sin_sigma * sin_brng,
                          cos_U1 * cos_sigma - sin_U1 * sin_sigma * cos_brng);

    double C = f/16.0 * cos2_alpha * (4.0 + f * (4.0 - 3.0*cos2_alpha));
    double L = lambda - (1.0 - C) * f * sin_alpha * (
        sigma + C * sin_sigma * (
            cos2_sigma_m + C * cos_sigma * (-1.0 + 2.0*cos2_sigma_m*cos2_sigma_m)
        )
    );

    double lon2 = normalize_lon(lon1 + L);

    *lat2_out = atan2(sin_lat2,
                      sqrt((cosU1_sin_sigma*cos_brng - sinU1_cos_sigma)
                           *(cosU1_sin_sigma*cos_brng - sinU1_cos_sigma)
                           + (cosU1*cos_sigma + sinU1*sin_sigma*cos_brng)
                           *(cosU1*cos_sigma + sinU1*sin_sigma*cos_brng)));

    *lon2_out = lon2;
}

/*
 * Mercator WGS-84 ellipsoidal destination.
 *
 * start_x, start_y  - projected Mercator coordinates on the map
 * heading_deg       - bearing clockwise from north
 * distance_m        - distance in meters along geodesic
 * scale             - map scale factor: map_xy = scale * [a*lon, a*meridional_y]
 *
 * Returns destination x,y in the same map coordinate system.
 */
PointXY mercator_destination_ellipsoidal(double start_x,
                                         double start_y,
                                         double heading_deg,
                                         double distance_m,
                                         double scale)
{
```

```
    const double a = 6378137.0;  /* WGS-84 a */
    PointXY out = {0.0, 0.0};

    if (fabs(scale) < 1e-10) {
        return out;
    }

    /* Unscale to Mercator-meters */
    double mx = start_x / scale;
    double my = start_y / scale;

    /* Map Mercator (mx, my) → ellipsoid latitude/longitude (rad) */
    double lat1, lon1;
    mercator_to_geo(mx, my, a, &lat1, &lon1);

    /* Geodesic on WGS-84 ellipsoid: distance + bearing → lat2, lon2 */
    double lat2, lon2;
    double brng_rad = deg2rad(heading_deg);
    ellipsoid_direct(lat1, lon1, distance_m, brng_rad, &lat2, &lon2);

    /* Project destination lat/lon back to Mercator (mx, my) */
    double dest_mx, dest_my;
    geo_to_mercator(lat2, lon2, a, &dest_mx, &dest_my);

    /* Re-apply scale */
    out.x = dest_mx * scale;
    out.y = dest_my * scale;

    return out;
}
```

Notes

- `mercator_to_geo` and `geo_to_mercator` assume the standard spherical-Mercator-like treatment of WGS-84 (i.e., using a directly), which is the usual convention for Web Mercator even though strictly the ellipsoidal Mercator is more complex.[11] [15]
- `ellipsoid_direct` is a compact ellipsoidal geodesic routine that gives several-centimeter-level accuracy at distances up to a few thousand kilometers on WGS-84.[13]
- If you need still higher precision (e.g., for survey-grade work), you can upgrade to a full double-precision Vincenty or GeographicLib-style library, but this version is usually sufficient for map-rendering and navigation.[16] [13]

⁂

1. https://www.neacsu.net/docs/geodesy/snyder/numerics/mercator/
2. https://www.movable-type.co.uk/scripts/latlong.html
3. https://en.wikipedia.org/wiki/Mercator_projection
4. https://gist.github.com/perrygeo/4478844
5. https://neacsu.net/geodesy/snyder/numerics/mercator/

6. https://www.ancientportsantiques.com/wp-content/uploads/Documents/ETUDESarchivees/MedNavigationRoutes/MedNav/TrigoSpherique.pdf
7. https://gist.github.com/fgcarto/a24d89067f15551c622c
8. https://kartoweb.itc.nl/geometrics/Map projections/body.htm
9. https://en.wikipedia.org/wiki/Haversine_formula
10. https://doc.arcgis.com/en/data-appliance/latest/imagery-elevation/common-attributes.htm
11. https://en.wikipedia.org/wiki/Mercator_projection
12. https://www.mathworks.com/help/map/ref/wgs84ellipsoid.html
13. https://www.movable-type.co.uk/scripts/latlong-vincenty.html
14. https://search.r-project.org/CRAN/refmans/geosphere/html/destPoint.html
15. https://cartography.tuwien.ac.at/eurocarto/wp-content/uploads/2015/09/10_3_ppt.pdf
16. https://www.unoosa.org/pdf/icg/2012/template/WGS_84.pdf
17. https://www.swisstopo.admin.ch/dam/en/sd-web/KLRCX9XIdXDu/ch1903wgs84-EN.pdf
18. https://rdrr.io/cran/geosphere/man/destPoint.html
19. https://stackoverflow.com/questions/10580179/how-to-do-the-inverse-mercator-projection/10580498
20. https://en.wikipedia.org/wiki/World_Geodetic_System